

Python — TUPLE (Part 1)

A tuple is an ordered collection of items.

Tuples are immutable: their items cannot be changed directly.

Tuples normally use parentheses ().

1. Creating a Tuple

```
numbers = (10, 20, 30, 40)
names = ("Jahanur", "Rahul", "Amit")
mixed = (10, "Python", True, 3.14)
empty = ()
```

2. Single-Item Tuple ☆

A comma is required for one item.

```
a = (10,)      # tuple
b = (10)       # integer, not tuple
type(a)        # <class "tuple">
```

3. Indexing

```
t = ("Python", "Java", "C++")
t[0]    # Python
t[1]    # Java
t[-1]   # C++
t[-2]   # Java
```

4. Slicing

Syntax: tuple[start : stop : step]

The stop index is not included.

```
t = (10, 20, 30, 40, 50, 60)
t[1:4]    # (20, 30, 40)
t[:3]     # (10, 20, 30)
t[::2]    # (10, 30, 50)
t[::-1]   # reverse tuple
```

Python — TUPLE (Part 2)

5. Tuple Unpacking ☆

Unpacking assigns tuple values to several variables.

```
person = ("Jahanur", 22, "India")
name, age, country = person
print(name)      # Jahanur
print(age)       # 22
```

6. Extended Unpacking

```
numbers = (1,2,3,4,5)
first, *middle, last = numbers
middle   # [2,3,4]
```

7. Tuple Methods

```
t = (10,20,20,30,20)
t.count(20)    # 3
t.index(30)    # 3
```

8. Membership & Length

```
t = (10,20,30)
20 in t        # True
50 not in t    # True
len(t)         # 3
```

9. Concatenation & Repetition

```
a=(1,2); b=(3,4)
a + b          # (1,2,3,4)
a * 3          # (1,2,1,2,1,2)
```

10. Nested Tuples

```
data = (("A",10), ("B",20), ("C",30))
data[1][0]    # B
data[2][1]    # 30
```

Python — TUPLE (Part 3)

11. Immutability — Important!

You cannot change, add, or remove tuple items directly.

```
t=(10,20,30)
t[1]=99      # ✗ TypeError
t.append(40)  # ✗ AttributeError
t.remove(20)  # ✗ AttributeError
```

12. How to 'Change' a Tuple

Create a new tuple, or convert it to a list temporarily.

```
t=(10,20,30)
temp=list(t)
temp[1]=99
t=tuple(temp)
print(t)      # (10,99,30)
```

13. Tuple with Different Types

```
item=("Laptop",16,74999.0,True)
str, int, float and bool can coexist.
```

14. Why Use Tuples?

- *Data should not change.*
- *Useful for fixed records and coordinates.*
- *Can be dictionary keys when all elements are hashable.*
- *They clearly communicate: 'this collection is fixed.'*

15. Tuple vs List

```
my_list  = [1,2,3]    # mutable
my_tuple = (1,2,3)    # immutable
# list: add/remove/change
# tuple: read/unpack/slice
```

Python — TUPLE (Part 4)

16. Tuple in Real Python Code

```
x, y = (10, 20)
x, y = y, x      # swap values
print(x,y)       # 20 10
```

17. Useful Built-ins

```
t=(4,1,9,2)
len(t)          # 4
min(t)          # 1
max(t)          # 9
sum(t)          # 16
sorted(t)       # [1,2,4,9]
```

18. DSA / Performance Idea

Tuple indexing is $O(1)$.

Searching with 'in' is $O(n)$.

Tuples are fixed-size/immutable, which can make them a good choice for fixed data.

19. Practice Programs ?

1. Find sum, minimum and maximum of a tuple.
2. Count occurrences of a value.
3. Find the index of a value.
4. Reverse a tuple.
5. Convert a list to a tuple.
6. Unpack a tuple into variables.
7. Swap two variables using tuple unpacking.
8. Find common elements of two tuples.
9. Access elements of a nested tuple.
10. Check whether an item exists in a tuple.

20. Quick Revision

Tuple = ordered + immutable collection.

Index starts at 0; negative index starts at -1.

One-item tuple needs a comma: (10,).

Main methods: count() and index().

Use tuples for fixed data; use lists for changing data.

Practice every concept with code — understanding comes from doing.